



Queues for Kafka 소개

Sangho Park (spark@confluent.io)
Senior Solutions Architect, IBM Confluent

2026-04-29

목차



1. Why Queues in Kafka?
2. Queues for Kafka 개요
3. Queues for Kafka 동작방식
4. 언제 **Queues for Kafka**를 사용하는가?
5. 결론



Why Queues in Kafka?



왜 **Kafka**에 "큐"가 필요했나?

기존 **Kafka** 소비 모델의 구조적 한계

파티션 = **Consume** 병렬성 단위

- 하나의 컨슈머 그룹에서 각 파티션은 정확히 한 컨슈머에게만 할당됨
- 병렬성을 늘리려면 파티션 수를 늘리는 것 외에는 선택지가 없음

이로 인해 생기는 패턴

- 최대 예상 트래픽을 기준으로 파티션 수를 잡게 되어 과도한 파티션(**over-partitioning**)을 사용하는 경우가 많음
- 파티션 수를 줄이는 것은 사실상 새로운 토픽으로 마이그레이션해야 해서 실제 운영에서 잘 하지 않음
- 토픽/파티션 수가 늘어날수록 메타데이터·리밸런스·모니터링 복잡도가 함께 증가

작업 큐(**work queue**) 관점에서의 요구사항

작업 큐에 흔한 시나리오

- 이미지/동영상 처리, 리포트 생성, 백그라운드 배치 등 서로 독립적인 작업 단위를 워커 풀에 던져 놓고 처리
- 워커 수를 부하에 따라 탄력적으로 늘렸다 줄였다 하며, 각 메시지는 워커 중 정확히 한 곳에서만 처리되면 됨

필요한 기능

- 메시지 단위 개별 재시도/승인/포기 (**ACK/RETRY/DROP**)
- 오래 걸리는 작업을 위해 처리 시간 연장(타임아웃 연장) 이 필요할 수 있음
- 처리 실패 메시지를 별도 토픽(**DLQ**)으로 보내서 나중에 재처리/분석

기존 **Kafka consumer group**으로 구현할 때의 어려움

- 오프셋은 파티션 단위로 커밋되기 때문에, 배치 중 일부만 실패한 경우 처리 로직이 복잡해짐
- 장시간 처리 작업에서 컨슈머가 죽거나 리밸런스되면, 어떤 레코드를 다시 처리해야 하는지 추적하기 까다로움



왜 **Kafka**에 "큐"가 필요했나?

전통적인 큐 시스템을 병행 운영할 때의 문제

많은 조직이 다음과 같이 아키텍처를 구성

- "실시간 스트리밍/이벤트 수집은 **Kafka**",
- "작업 큐/태스크 분배는 **RabbitMQ, SQS**, 기타 큐 서비스"

이 방식의 비용

- 이중 인프라 운영: 모니터링, 백업, 보안/컴플라이언스, 업그레이드 작업을 두 번 수행
- 애플리케이션이 **Kafka** ↔ 큐 시스템 사이를 전환할 때 데이터 모델 변환, 중복 로깅, 트랜잭션 경계 처리 등 추가 복잡도 발생
- 장애 분석 시, 이벤트가 **Kafka**에서 큐로 넘어가는 경로를 추적해야 해 관측 가능성 (**observability**)이 떨어짐

우회 패턴의 한계

Parallel Consumer 등 다양한 우회 패턴이 존재 하지만 공통적인 한계

- 구현이 복잡하고, 팀마다 패턴이 달라 표준화/재사용이 어려움
- 브로커 레벨이 아니라 클라이언트 라이브러리/애플리케이션 코드 레벨에서 동작하기 때문에, 운영·관리가 쉽지 않음
- **per-message** 상태(재시도 횟수, 실패 이유 등)에 대한 일관된 관측·제어가 부족



왜 **Kafka**에 "큐"가 필요했나?

Kafka는 이미 스트리밍에는 최적화되어 있지만, 작업 큐/태스크 분배 관점에서 필요한 기능이 표준화되어 있지 않았고, 이를 위해 별도 큐 인프라를 운영하거나 복잡한 우회 솔루션에 의존해야 했다. 그래서 **Kafka** 자체에 큐 방식을 지원하는 **Queues for Kafka**가 필요해졌다.



Queues for Kafka 개요



Queues for Kafka 개요

- KIP-932: Queues for Kafka – Kafka에 큐 방식을 지원하는 프로토콜 확장
- 핵심 개념
 - Share Group
 - 기존 consumer group의 대안으로, 여러 컨슈머가 같은 파티션을 협력적으로(consumption sharing) 소비
 - 파티션 수보다 많은 컨슈머 실행 가능 → 파티션 수와 무관하게 워커 수를 탄력적으로 조정
 - Share Partition
 - 브로커 측에 도입된 새로운 논리적 구조로, 실제 데이터가 담긴 원본 토픽 파티션을 가리키는 '포인터' 역할.
 - 원본 데이터를 복제하지 않고 참조만 하는 뷰 개념
 - KafkaShareConsumer API
 - 메시지 단위 개별 ACCEPT/RELEASE/REJECT/RENEW 지원으로, 작업 큐에 필요한 메시지 라이프사이클 제어 제공
- 결과
 - Kafka 하나로 스트리밍 + 큐 워크로드를 통합 운영 가능



Kafka 토픽 vs 전통적인 큐

- 전통 큐
 - 메시지 소비 시 **파괴적 읽기 (read-and-delete)** → 다른 컨슈머는 동일 메시지 재처리 불가
 - **재처리·리플레이**가 어렵고, 영속성 보다는 작업 처리에 초점
- Kafka 토픽
 - 기본은 **로그(log)**: 읽어도 레코드는 토픽에 그대로 남아 다수 컨슈머가 독립적으로 읽을 수 있음
 - **리텐션설정 및 Compaction**으로 장기간 데이터 보관 및 리플레이 가능
- Queues for Kafka의 접근
 - 토픽은 그대로 유지하되, **소비 모델**만 share group + share consumer로 큐처럼 동작하게 함

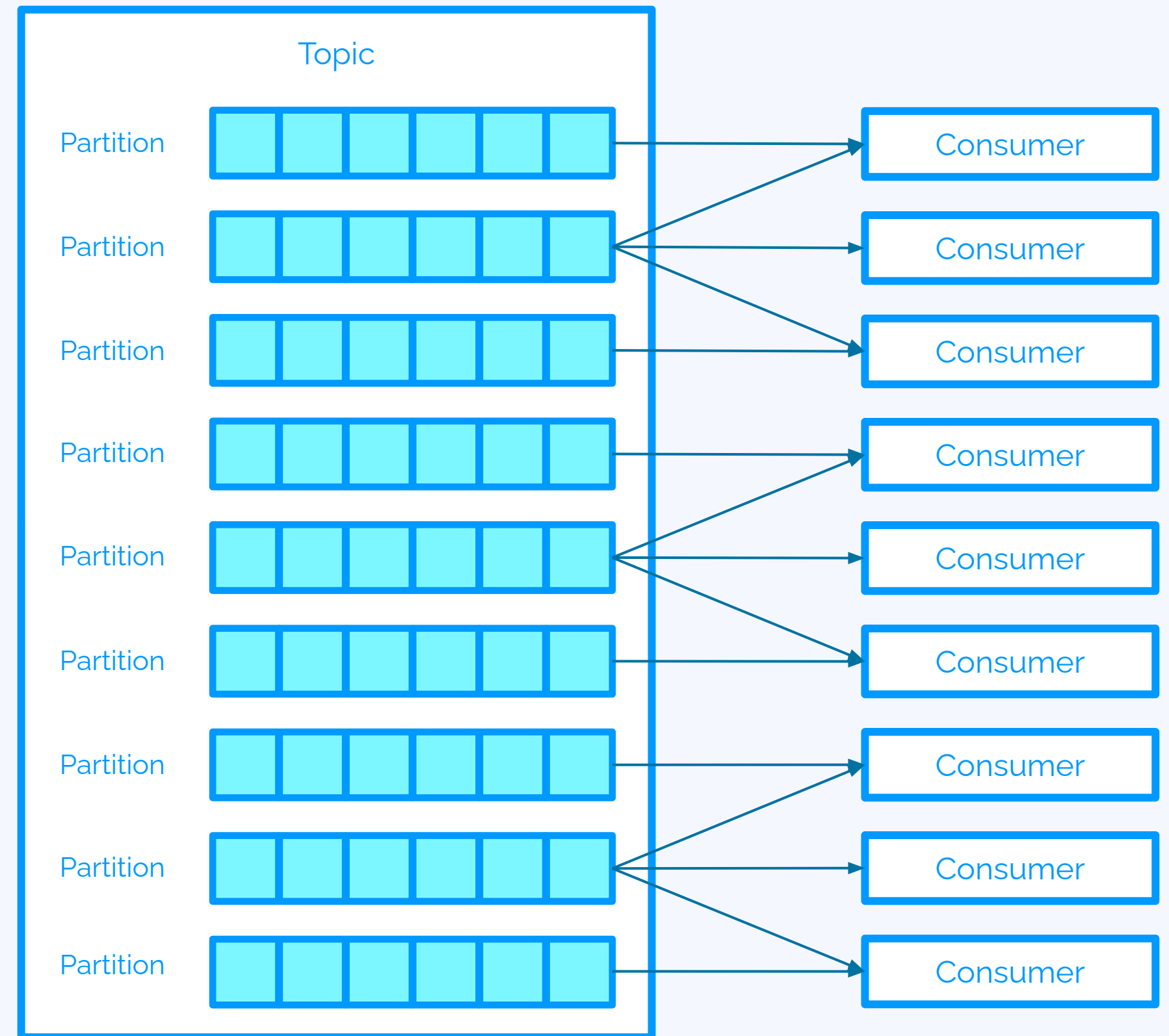


Consumer Group vs Share Group 구조 비교

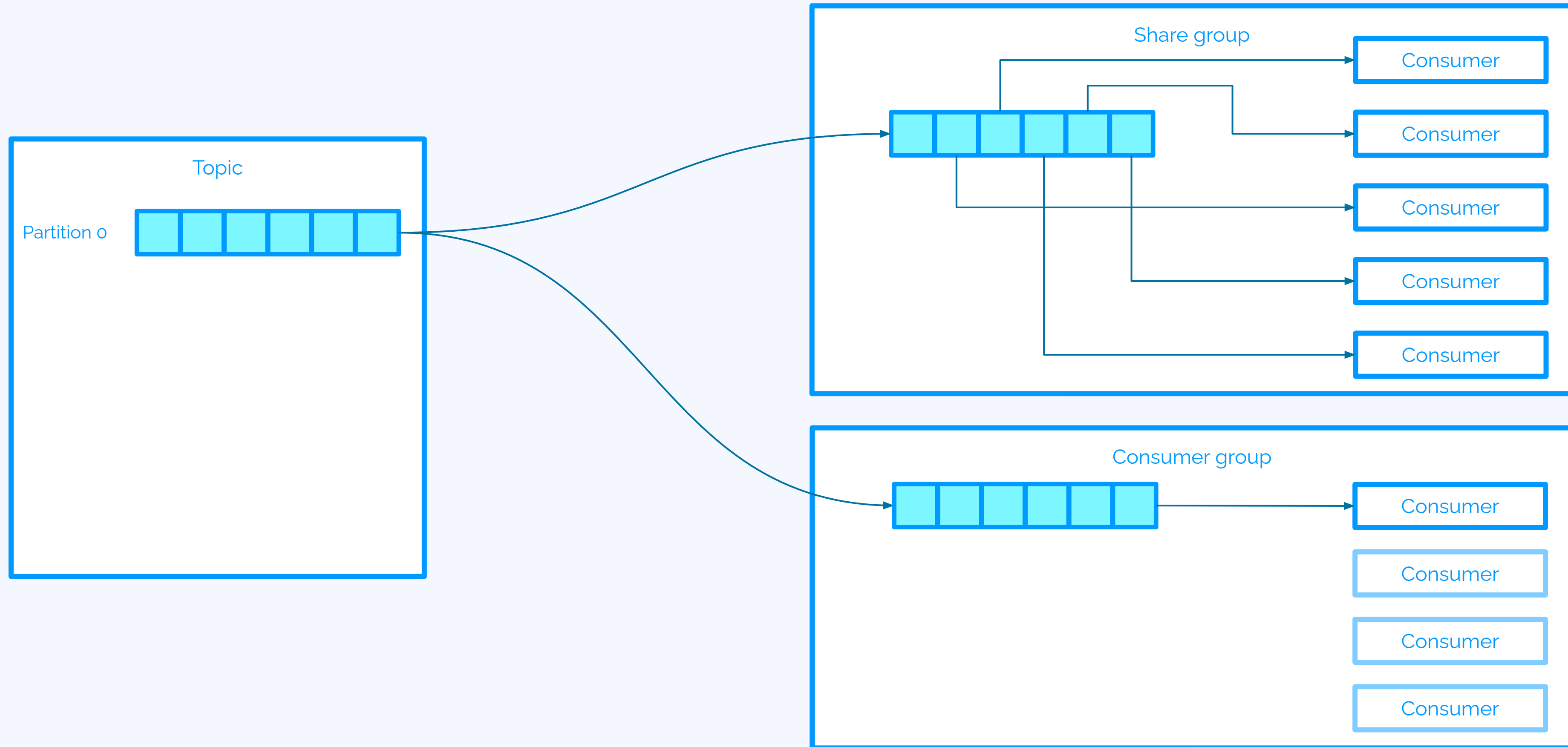
- **기존 Consumer Group**
 - 각 파티션은 그룹 내 **정확히 한 컨슈머**에게만 할당
 - 파티션 수가 병렬처리의 상한선을 형성 → 피크 부하에 맞게 파티션 수를 설정해야 함
- **Share Group (Queues for Kafka)**
 - 여러 컨슈머가 **동일 파티션의 레코드를 분담**해서 처리 (cooperative consumption)
 - **컨슈머 수와 파티션 수의 결합이 풀림** → 워커 풀을 자유롭게 오토스케일링 가능
 - 내부적으로는 레코드 단위 락·상태 머신으로 "한 레코드는 한 번에 한 컨슈머만" 처리되도록 보장

사용 사례 - 피크 부하를 처리하기 위해 컨슈머를 확장

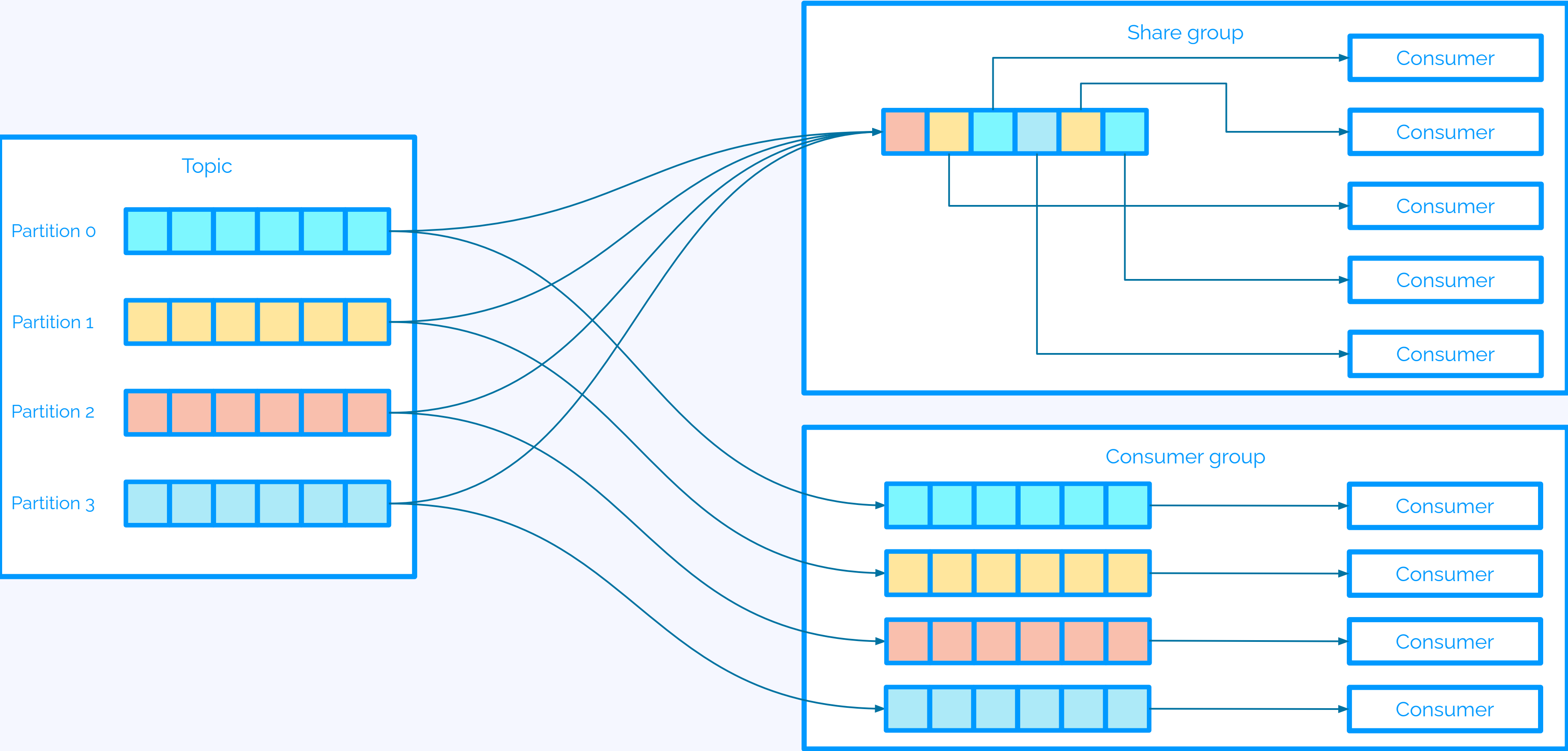
- 특정 시간대에 메시지 인입 속도가 급격히 높아지는 현상이 발생한다고 했을때
- 처리 지연을 목표치 이내로 통제하려면 해당 시간대에는 더 많은 수의 컨슈머가 가동되어야 합니다.
- 이를 해결하기 위해 최대 부하 기준에 맞춰 파티션을 과다 할당(Over-partition)하는 방식을 택하시겠습니까?
- 아니면 적체된 메시지가 모두 처리될 때까지 한시적으로 컨슈머를 스케일 아웃(Scale out)하는 방식을 사용하시겠습니까?



하나의 파티션을 컨슈머들이 공유



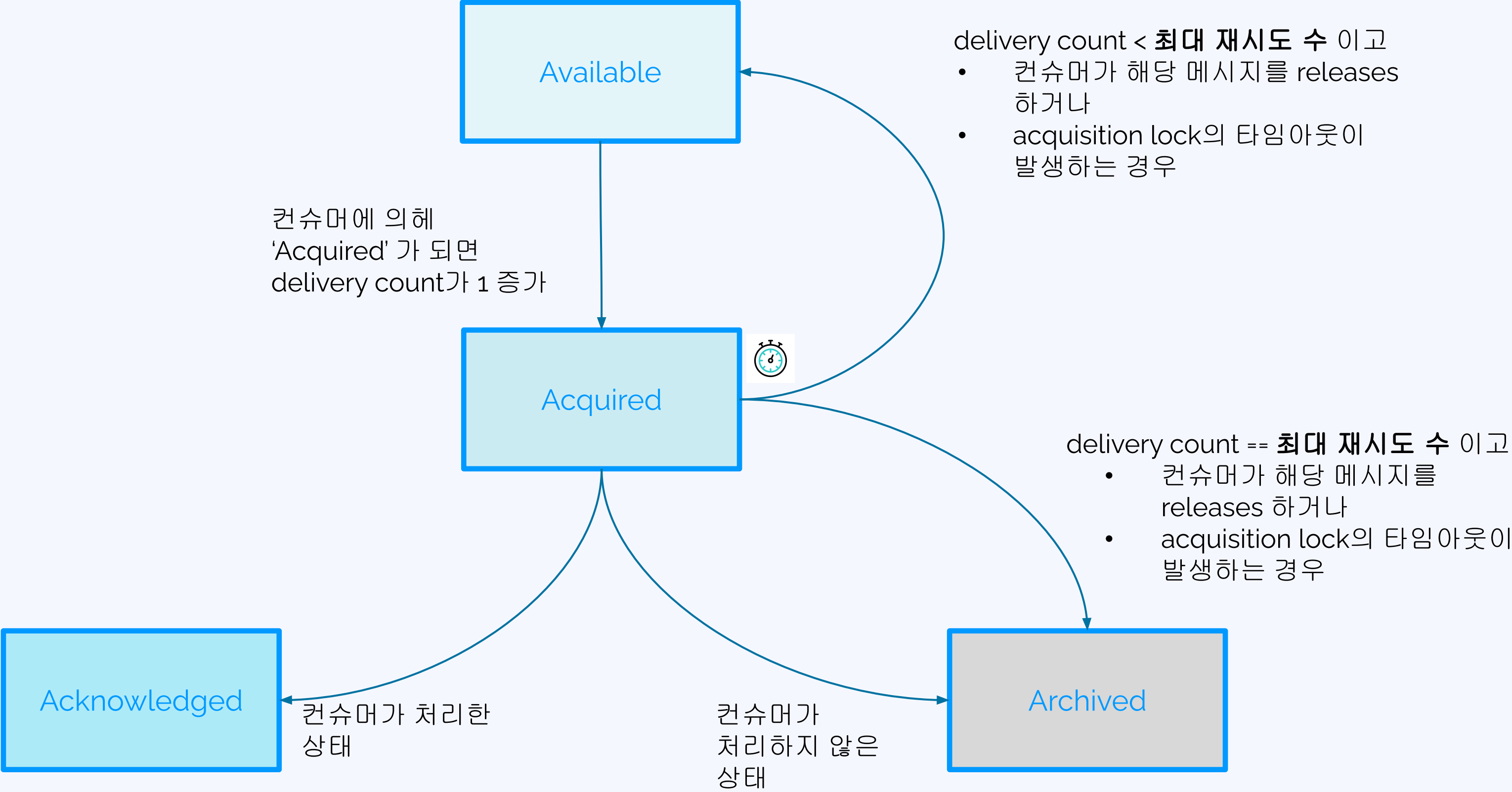
복수의 파티션을 컨슈머들이 공유





Queues for Kafka 동작방식

메시지 라이프사이클 & 상태





메시지 라이프사이클 & 상태

- Share partition : (특정 share group 입장에서 본) topic-partition + 그 그룹 전용의 메시지 상태/락/재시도 정보를 합친 논리적 단위
- Share-partition 내 레코드 상태
 - **Available**: 토픽에 존재하며 아직 어떤 컨슈머에 의해 잡히지 않은 상태
 - **Acquired**: 특정 컨슈머가 시간 제한 락을 잡은 상태, 다른 컨슈머에게는 보이지 않음
 - **Acknowledged**: 성공 처리 후 ACK된 상태
 - **Archived**: 최대 재시도 횟수 초과 혹은 REJECT 등으로 더 이상 소비될 수 없는 상태
- Share-partition 내 처리 중인 레코드 창 - In-flight Records Window
 - Share Partition은 메시지 처리를 추적하기 위해 '창(Window)' 개념을 사용
 - 가장 처음 나타나는 **Available** 또는 **Acquired** 메시지의 위치를 **Start Offset**으로 지정하고, 설정된 개수만큼의 크기를 가지는 창 안에서만 데이터를 처리
 - **Start Offset**의 메시지가 최종 상태(**Acknowledged** 또는 **Archived**)로 변경되면, 이 처리 창이 앞으로 전진
- Delivery count
 - 각 레코드에는 **delivery count**가 있어 몇 번 시도했는지 추적
- 관련 설정
 - `group.share.delivery.count.limit` 로 최대 재시도(기본 5회) 설정
 - `group.share.record.lock.duration.ms` 로 레코드 락 유지 시간 (기본 30초) 설정
 - `group.share.partition.max.record.locks` 공유 파티션(Share-partition)당 동시에 처리 중인(In-flight) 레코드의 최대 개수를 정의합니다. 기본값은 2000입니다.

프로토콜

ShareGroupHeartbeat (그룹 멤버십 및 할당)

컨슈머 리밸런스 프로토콜(KIP-848)의 아키텍처를 기반으로 동작하는 Share Group 전용 하트비트 프로토콜

- 컨슈머가 Share Group에 가입하거나 유지하고, 브로커(Group Coordinator)로부터 파티션을 할당받는 데 사용됩니다.
- 컨슈머는 주기적으로 **ShareGroupHeartbeat** 요청을 보내 자신이 살아있음을 알리고, 브로커는 이 응답(Response)에 컨슈머가 읽어야 할 파티션 목록(Assignment)을 할당

ShareFetch (메시지 획득 및 잠금)

컨슈머가 브로커(Share Partition Leader)로부터 데이터를 가져옴

- 메시지를 가져옴과 동시에, 브로커 측의 메시지 상태를 변경하고 잠금(Lock)을 겁니다.
- **ShareFetch** 요청 시, 컨슈머는 이전에 처리 완료한 메시지에 대한 **응답(Acknowledge)** 데이터도 함께 실어서 보낼 수 있습니다. 이를 통해 네트워크 통신 횟수를 줄입니다.

ShareAcknowledge (상태 응답)

컨슈머가 가져간 레코드들의 처리 결과를 브로커에 통보하는 프로토콜입니다.

- 오프셋 기반의 커밋(OffsetCommit)을 대체하며, 개별 메시지나 배치의 처리 상태를 확정합니다.
- **AcknowledgeType**
 - **ACCEPT (1):** 성공적으로 처리 완료. 브로커는 상태를 'ACKNOWLEDGED'로 변경하고 윈도우를 전진
 - **RELEASE (2):** 일시적 오류 발생. 잠금을 즉시 풀고 상태를 다시 'AVAILABLE'로 돌려 다른 컨슈머가 가져갈 수 있도록 함
 - **REJECT (3):** 영구적 오류 발생. 메시지를 폐기 처리(ARCHIVED)합니다.
- 백그라운드에서 단독으로 전송되거나, 다음 **ShareFetch** 요청에 피기백(Piggyback)되어 전송됩니다.



KafkaShareConsumer API – 암시적 (implicit) ACK

- 단순한 작업 큐에 적합한 모드
- 아이디어: 이전 `poll()`에서 받은 배치를, 다음 `poll()` 또는 `commit` 시 한 번에 **ACCEPT** 처리

```
Properties props = new Properties();
props.setProperty("bootstrap.servers", "localhost:9092");
props.setProperty("group.id", "myshare");
```

```
KafkaShareConsumer<String, String> consumer =
    new KafkaShareConsumer<>(props, new StringDeserializer(), new StringDeserializer());
```

```
// Share Group 참여
consumer.subscribe(Arrays.asList("work-queue"));

while (true) {
    // 새 배치를 가져오면서, 이전 배치는 암시적으로 ACCEPT 처리
    ConsumerRecords<String, String> records =
        consumer.poll(Duration.ofMillis(100));

    for (ConsumerRecord<String, String> record : records) {
        doProcessing(record); // 비즈니스 로직
    }
}
```

- 장점: 코드가 간단하고 **idempotent** 처리에 적합
- 단점: 배치 내 개별 레코드 단위의 세밀한 오류 처리에는 부적합



KafkaShareConsumer API – 명시적(explicit) ACK

- 레코드마다 ACCEPT / RELEASE / REJECT (및 RENEW 등) 를 명시적으로 지정

```
Properties props = new Properties();
props.setProperty("bootstrap.servers", "localhost:9092");
props.setProperty("group.id", "myshare");
props.setProperty("share.acknowledgement.mode", "explicit");

KafkaShareConsumer<String, String> consumer =
    new KafkaShareConsumer<>(props, new StringDeserializer(), new StringDeserializer());

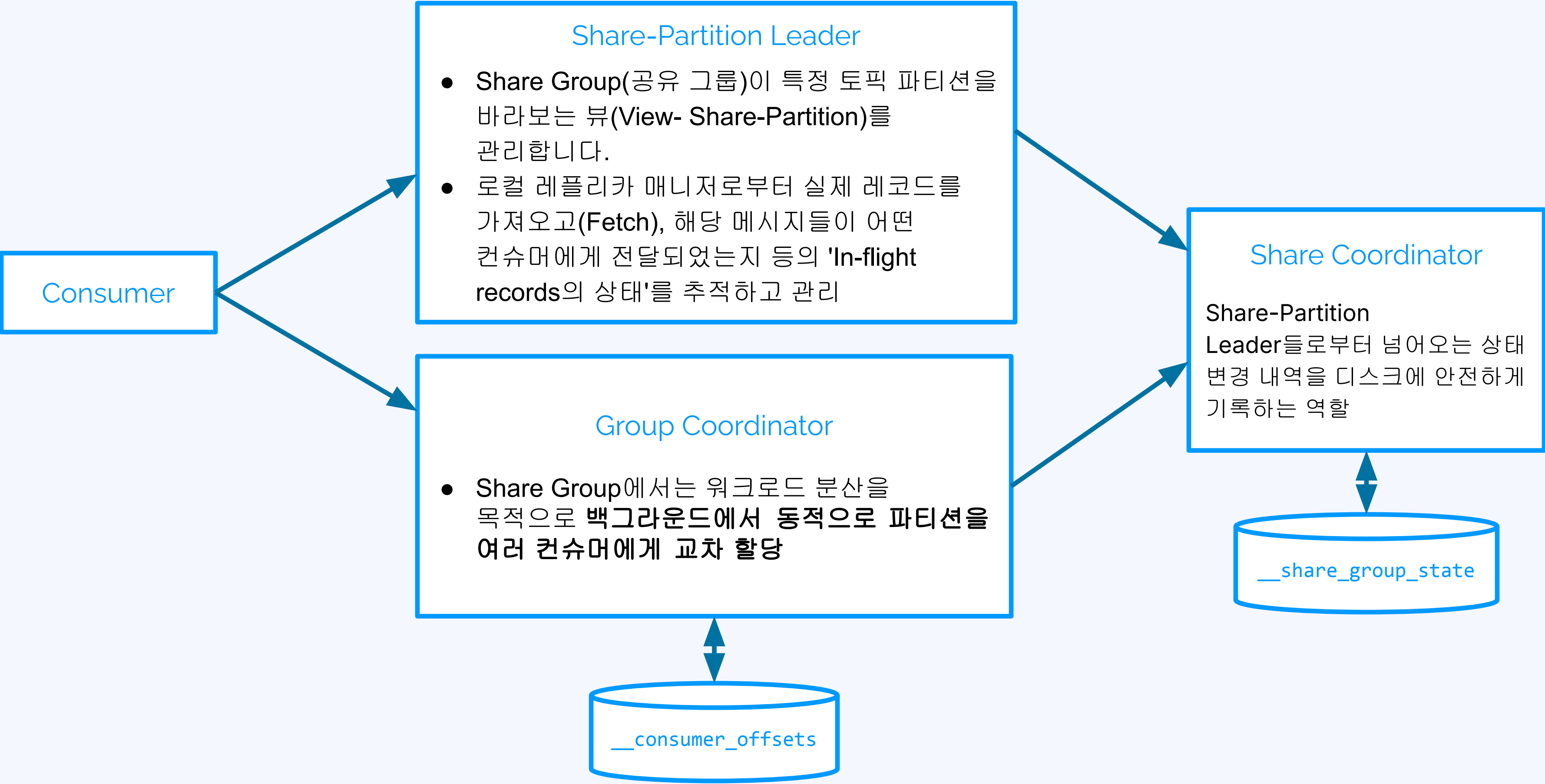
consumer.subscribe(Arrays.asList("work-queue"));

while (true) {
    ConsumerRecords<String, String> records =
        consumer.poll(Duration.ofMillis(100));

    for (ConsumerRecord<String, String> record : records) {
        try {
            doProcessing(record);
            consumer.acknowledge(record, AcknowledgeType.ACCEPT); // 성공
        } catch (TransientException e) {
            consumer.acknowledge(record, AcknowledgeType.RELEASE); // 재시도
        } catch (FatalException e) {
            consumer.acknowledge(record, AcknowledgeType.REJECT); // DLQ 전송 대상
            sendToDLQ(record); // DLQ 전송
        }
    }
    consumer.commitAsync();
}
```

- 복잡한 워크플로우, **DLQ 연계** 패턴에 적합하나 현재는 직접적으로 구현해야 함 (현재 KIP-1191: Share Group을 위한 데드 레터 큐 기능 구현 기능을 Apache Kafka 4.4에 추가되는것을 목표로 진행 중)

동작 방식



사용 방법

- `kafka-console-share-consumer.sh` : share group에 있는 데이터 소비
- `kafka-share-consumer-perf-test.sh` : share groups 성능 테스트
- `kafka-groups.sh` : 모든 유형의 그룹 관리
- `kafka-share-groups.sh` : share groups 관리 명령어



언제 **Queues for Kafka**를 사용하는가?



대표 유즈케이스

- **작업 큐 / 워커 풀**
 - 이미지 처리, 리포트 생성, ML inference 등 **독립 작업 단위**를 워커 풀에 분배
- **알림·이벤트 처리**
 - 모니터링 알림, 마케팅 캠페인 발송 등, 트래픽 스파이크 시 워커를 **탄력적으로 스케일 아웃**해서 SLA 유지
- **Fraud Detection / Risk 분석**
 - 트랜잭션 단위 분석을 여러 모델/워커에 분산 처리, 부하에 따라 동적으로 컨슈머 수 조정
- **백그라운드 비즈니스 작업**
 - 주문 후처리, 결제 정산, 이메일 발송 등 **비동기 업무 로직**을 Kafka로 통합해 하나의 플랫폼에서 운영



Queues for Kafka를 언제 써야 할까?

- **Share Group(Queues for Kafka)를 추천하는 경우**
 - 파티션 수와 무관하게 워커 수를 자유롭게 조정해야 하는 작업 큐(work queue) 패턴
 - 메시지별 개별 재시도/포기/연장(RENEW)이 중요한 경우
 - 기존 별도 큐 시스템을 Kafka로 통합해 **운영 단순화·TCO 절감**이 목표인 경우
- **기존 Consumer Group이 더 적합한 경우**
 - 파티션 단위 엄격한 순서 보장(ordering)이 필수인 스트리밍 파이프라인
 - 같은 이벤트를 여러 애플리케이션이 **모두 읽어야 하는 fan-out 스트리밍** 시나리오
- 설계 시 추가 고려사항
 - **DLQ(topic)** 설계와 모니터링 전략
 - Share lag, in-flight 레코드 수, 재시도율 등 **새로운 지표 기반 모니터링** 필요



결론

결론

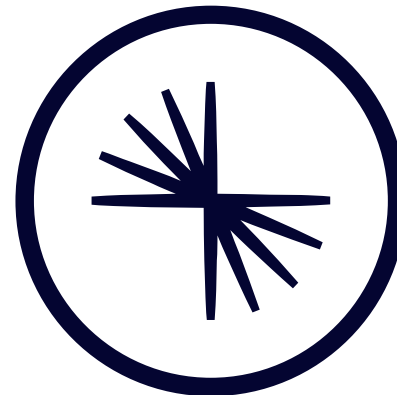
- Queues for Kafka는
 - Kafka에 네이티브 큐 개념 추가해, 스트리밍 + 큐 워크로드를 하나의 플랫폼에 통합
 - **Share Group + KafkaShareConsumer**로 파티션 수와 무관한 탄력적 소비와 메시지 단위 제어 제공
- 기대 효과
 - 별도 큐 시스템 제거 또는 축소 → 운영 단순화, 비용 절감, 데이터 일관성 향상
 - 작업 큐, 알림, 백그라운드 처리, 에이전트 워크플로 등 새로운 실시간 유즈케이스 확장
- 한 줄 요약
 - **"Kafka를 진짜 큐처럼 쓰고 싶다면, 이제는 Share Group을 쓰면 된다."**

Get started with Queues for Kafka today!

Available on Confluent Cloud now! (Enterprise and Dedicated Clusters)



Start your [free trial](#) of
Confluent Cloud



*New signups receive \$400 to spend during
their first 30 days—no credit card required.*

[***https://www.confluent.io/get-started/***](https://www.confluent.io/get-started/)

